

Vision Based Adaptive Traffic Signal Control

**A project report submitted in partial fulfillment of the requirement for the
Award of the Degree of**

BACHELOR of ENGINEERING

in

Electronics and Communication Engineering

by

Umme Hafeeza Sultana, 160722735068

Bathula Vennela, 160722735070

Gajula Sathvika, 167227350102

Under the Guidance of

Dr. Ravi M Yadahalli

Professor, Dept of ECE



Estd : 2008

**Department of Electronics and Communication Engineering
Methodist College of Engineering and Technology (Autonomous)**

King Koti, Abids, Hyderabad – 500001

2025-26

Methodist College of Engineering and Technology (Autonomous)

King Koti, Abids, Hyderabad-500001

Department of Electronics and Communication Engineering



Estd : 2008

DECLARATION BY THE CANDIDATES

We, Umme Hafeeza Sultana, 160722735068, Bathula Vennela, 160722735070 and Gajula Sathvika, 160722735102 students of Methodist College of Engineering and Technology, (Autonomous) pursuing Bachelor's degree in Electronics & Communication Engineering, hereby declare that this Project report entitled "Vision Based Adaptive Traffic Signal Control", carried out in 2025-26 under the guidance of Dr. Ravi M Yadahalli, Professor, Dept of ECE submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Electronic Communication Engineering. This is a record work carried out by us, and the results embodied in this report have not been reproduced/copied from any source.

Umme Hafeeza Sultana, 160722735068

Bathula Vennela, 160722735070

Gajula Sathvika, 160722735102

Methodist College of Engineering and Technology (Autonomous)

King Koti, Abids, Hyderabad – 500001

Department of Electronics and Communication Engineering



Estd : 2008

CERTIFICATE BY THE SUPERVISOR

This is to certify that this Dissertation on project work entitled “Vision Based Adaptive Traffic Signal Control” by, Umme Hafeeza Sultana, 160722735068, Bathula Vennela, 160722735070 and Gajula Sathvika, 160722735102 submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Electronics & Communication Engineering, during the academic year 2025-26, is a bonafide record of work carried out by them.

Dr. Ravi M Yadahalli
Professor, ECE

Date:

Methodist College of Engineering and Technology (Autonomous)

King Koti, Abids, Hyderabad-500001

Department of Electronics and Communication Engineering



Estd : 2008

CERTIFICATE BY HEAD of THE DEPARTMENT

This is to certify that this Dissertation on project work entitled “Vision Based Adaptive Traffic Signal Control” by Umme Hafeeza Sultana, 160722735068, Bathula Vennela, 160722735070, Gajula Sathvika, 160722735102 submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Electronics & Communication Engineering of the Osmania University, Hyderabad, during the academic year 2025-26, is a bonafide record of work carried out by them.

Dr. VSSN SRINIVASA BABA
Professor & Head of the Department, ECE

Date:

Methodist College of Engineering and Technology (Autonomous)

King Koti, Abids, Hyderabad-500001

Department of Electronics and Communication Engineering



Estd : 2008

PROJECT APPROVAL CERTIFICATE

This is to certify that this Dissertation on project work entitled “Vision Based Adaptive Traffic Signal Control” by Umme Hafeeza Sultana, 160722735068, Bathula Vennela, 160722735070, Gajula Sathvika, 160722735102 submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Electronics & Communication Engineering of the Osmania University, Hyderabad, during the academic year 2025-26, is a bonafide record of work carried out by them.

Internal Examiner

External Examiner

HOD, ECE

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to my Project guide Dr. Ravi M Yadahalli., Professor, ECE, for giving us the opportunity to work on this topic. It would never be possible for us to take this project to this level without his innovative ideas and his relentless support and encouragement.

We would like to thank our project coordinator Dr. Ravi M Yadahalli, Professor, ECE, who helped us by being an example of high vision and pushing towards greater limits of achievement.

Our sincere thanks to Dr. VSSN Srinivasa Baba, Professor and Head of the Department of ECE, for his valuable guidance and encouragement which has played a major role in the completion of the project and for helping us by being an example of high vision and pushing towards greater limits of achievement.

We would like to express a deep sense of gratitude towards the Dr. Prabhu G. Benakop, Principal, Methodist College of Engineering and Technology, (Autonomous), for always being an inspiration and for always encouraging us in every possible way.

We would like to express a deep sense of gratitude towards the Dr. Lakshmipathi Rao, Director, Methodist College of Engineering and Technology (Autonomous), for always being an inspiration and for always encouraging us in every possible way.

We are indebted to the Department of Electronics & Communication Engineering and Methodist College of Engineering and Technology (Autonomous), for providing us with all the required facility to carry our work in a congenial environment. We extend our gratitude to the ECE Department staff for providing us to the needful time to time whenever requested.

We would like to thank our parents for allowing us to realize our potential, all the support they have provided us over the years was the greatest gift anyone has ever given us and also for teaching us the value of hard work and education. Our parents have offered us with tremendous support and encouragement, thanks to our parents for all the moral support and the amazing opportunities they have given us over the years.

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

Vision & Mission

VISION

To strive to become center of excellence in Education, Research with moral, ethical values and serve society

MISSION

- M1:** To provide Electronics & Communication Engineering knowledge for successful career either in industry or research
- M2:** To develop Industry-Interaction for innovation, product-oriented research and development.
- M3:** To facilitate value added education combined with hands-on training

Program Educational Objectives

Graduates of Electronics and Communication at Methodist College of Engineering and Technology will be able to:

- PEO1:** Apply the knowledge of Basic sciences and Engineering in designing and implementing the solutions in emerging areas of Electronics and Communication Engineering
- PEO2:** Pursue the research or higher education and practice profession
- PEO3:** Adapt to the technological advancements for providing the sustainable engineering solutions to meet organization/society needs
- PEO4:** Work as an individual or in a team with professional ethics and values

PROGRAM SPECIFIC OUTCOMES

At the end of 4 years, Electronics and Communication graduates at MCET will be able to:

- PSO1:** Apply the knowledge of Electronics & Communication Engineering principles in VLSI, Signal processing, Communication, Embedded system & Control Engineering
- PSO2:** Design and implement products using the cutting- edge software and hardware tools
- PSO3:** Demonstrate the leadership qualities and strive for the betterment of organization, environment and society

PROGRAM OUTCOMES

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society, and environment as required.

PO4: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions.

PO5: Engineering tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The engineer and The World: Analyse and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.

PO8: Individual and collaborative teamwork: Function effectively as an individual, and as a member or leader in diverse teams/multi-disciplinary teams.

PO 9: Communication: Communicate effectively and inclusively within engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, considering cultural, language, and learning differences.

PO10: Project management and finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-long learning: Recognize the need for and have the preparation and ability for

- i). Independent and life-long learning
- ii). Adaptability to new and emerging technologies
- iii). Critical thinking in the broadest context of technological change.

ABSTRACT

This study proposes a vision-based adaptive traffic light control system that utilizes real-time video analysis to intelligently manage urban intersections. By employing computer vision techniques with OpenCV, vehicles are detected, classified, and counted across multiple lanes, enabling accurate estimation of congestion levels. Based on these measurements, signal timings are dynamically adjusted, with green light durations proportionally assigned to the traffic density of each lane, thereby reducing idle waiting times and improving throughput. A lane prioritization mechanism ensures that heavily congested lanes receive precedence, while an emergency override feature allows immediate recognition and passage of critical vehicles such as ambulances and fire trucks, significantly enhancing response times for essential services. The system is implemented on embedded hardware platforms such as the ESP32, offering a cost-effective and scalable solution suitable for smart city infrastructure. By integrating adaptive timing, congestion-aware prioritization, and emergency vehicle recognition, the proposed framework contributes to more efficient traffic distribution, reduced delays, and improved safety, ultimately supporting the development of intelligent transportation systems in modern urban environments.

Beyond immediate traffic optimization, the proposed system demonstrates significant potential for integration into broader smart city ecosystems. By coupling vision-based traffic control with Internet of Things (IoT) connectivity, cloud-based data analytics, and predictive artificial intelligence models, intersections can evolve into intelligent nodes that not only respond to current traffic conditions but also anticipate future congestion patterns. Such integration enables coordinated traffic management across multiple junctions, supports data-driven urban planning, and facilitates seamless communication with emergency response networks. Moreover, the use of lightweight embedded platforms such as the ESP32 ensures scalability and cost-effectiveness, making the solution adaptable for both developing and developed urban infrastructures. This holistic approach positions the system as cornerstone for next-generation intelligent transportation systems, contributing to safer roads, reduced emissions, and enhanced commuter experiences.

Table of Contents

Sno.	Title	Page no.
1.	Introduction 1.1 Overview of Traffic Congestion and Road Safety 1.1.1 Need for Vision-Based Traffic System 1.1.2 Limitations of Existing System 1.2 Technological Evolution in Intelligent Traffic Systems 1.3 Role of open-cv in intelligent traffic control systems 1.4 Importance of Adaptive traffic control systems 1.5 Challenges in Detecting Traffic Conditions 1.6 Applications 1.7 Summary	1-7
2.	Literature Review 2.1 Literature Survey 2.1.1 ESP32-CAM Based Solutions in Literature 2.1.2 YOLO-Based Vehicle Detection Research 2.1.3 Classical Vision-Based Methods 2.2 Gap Analysis 2.2.1 Proposed Work 2.3 Objectives 2.4 Problem Statement	8-14
3.	Design Analysis 3.1 Model Architecture 3.1.1 Proposed System Architecture 3.1.2 System Components Description 3.1.3 Data Flow & Functional Workflow 3.1.4 Design Considerations & Constraints	15-19
4.	Modules and Implementation Module Descriptions 4.1.1 Input and pre-processing Module 4.1.2 Vehicle Detection and Classification Module 4.1.3 Traffic Density Estimation Module 4.1.4 Control and IoT Actuation Module 4.1.5 Emergency Vehicle Detection Module 4.1.6 IoT Monitoring and Dashboard Module 4.2 Module Interaction Diagram 4.3 Activity Diagram 4.4 Sequence Diagram 4.5 Summary	4.1 20-24

5.	Testing 5.1 System Testing 5.2 Types of Testing 5.2.1 Unit Testing 5.2.2 Integration Testing 5.2.3 Functional Testing 5.2.4 System Testing 5.2.5 Acceptance Testing 5.2.6 Summary	25-30
6.	Results and Analysis	31-35
7.	Conclusion 7.1 Performance Metrics	36
8.	References	37
9.	Appendix A. Sample Code B. Hardware and Software Requirements C. Quality Analysis and Alignment with POs and SDGs	38-45

LIST OF FIGURES

S. No	Figure No.	Figure Name/Description	Page No.
1	Figure 3.1	Data flow of vision based adaptive traffic control systems	18
1	Figure 4.2	Module Interaction Architecture of the vision based adaptive traffic control systems	
2	Figure 4.3	Operational Activity Diagram for vision based adaptive traffic control systems	22
3	Figure 4.4	Sequence Diagram of the Intelligent vision based adaptive traffic control systems	23
4	Figure 6.1	Detection, count and density of vehicles on a road also identifying the type of vehicle	31
5	Figure 6.2	Detection of vehicles on the congested road	32
6	Figure 6.3	Detecting the emergency vehicle by using YOLOv8	32
7	Figure 6.4	Traffic flow metrics	33
8	Figure 6.5	Correlation between traffic density and signal timing	34
9	Figure 6.6	Emergency vehicle appearance rush for green signal	35

LIST OF TABLES

S. No	Table No.	Table Title/Description	Page No.
1	Table 5.1	Unit Testing Results	26
2	Table 5.2	Integration Testing Results	27
3	Table 5.3	System testing	28
4	Table 5.4	Functional Testing Results	2
5	Table 5.5	Acceptance Testing Results	29
6	Table 5.6	Test Summary	29

1. Introduction

Rapid urbanization and the exponential growth of vehicle ownership have intensified traffic congestion, making it one of the most critical challenges for modern cities. Conventional traffic signal systems operate on fixed cycles, which fail to adapt to fluctuating traffic conditions and often result in unnecessary delays, wasted fuel, and increased pollution. These inefficiencies affect daily commuters, students, and elderly citizens alike, while emergency vehicles such as ambulances, fire engines, and police vans face life-threatening delays at intersections. The inability of traditional systems to respond dynamically to real-time traffic demands highlights the urgent need for intelligent, adaptive solutions that can improve mobility, reduce environmental impact, and enhance public safety.

Advancements in computer vision, artificial intelligence, and embedded systems provide a promising pathway to address these challenges. Video-based traffic monitoring using OpenCV enables real-time vehicle detection, lane occupancy estimation, and emergency vehicle recognition without the need for costly sensor infrastructure. When combined with lightweight deep learning models such as YOLOv8 or YOLOv4-tiny, these systems achieve high-speed and accurate detection suitable for real-time applications. Furthermore, the ESP32 microcontroller offers a cost-effective, energy-efficient, and IoT-enabled platform for implementing adaptive traffic control. By integrating vision-based detection with dynamic signal timing and emergency overrides, the proposed system not only enhances traffic efficiency but also supports the development of scalable smart city infrastructure, making it a compelling solution for the future of urban transportation.

1.1 Overview of Traffic Congestion and Road Safety

The primary objective of this project is to design and implement a vision-based adaptive traffic signal control system that can intelligently manage congestion in real time. By leveraging computer vision techniques with OpenCV, the system aims to detect and count vehicles across multiple lanes, estimate lane occupancy, and dynamically adjust green-light durations in proportion to traffic density. This adaptive approach ensures that heavily congested lanes receive priority, thereby reducing waiting times, improving throughput, and minimizing unnecessary fuel consumption and emissions. In addition, the system incorporates

emergency vehicle recognition, enabling ambulances, fire engines, and police vans to bypass intersections without delay, which is critical for saving lives and improving public safety.

Another key objective is to develop a cost-effective and scalable solution suitable for smart city infrastructure. The project utilizes the ESP32 microcontroller, which offers an optimal balance of processing capability, low power consumption, and built-in Wi-Fi connectivity, making it ideal for IoT-enabled traffic automation. By integrating lightweight deep learning models such as YOLOv8 or YOLOv4-tiny, the system achieves high-speed and accurate detection while remaining efficient enough for real-time deployment. Ultimately, the project seeks to demonstrate how adaptive signal timing, lane prioritization, and emergency overrides can be combined into a single framework that enhances traffic efficiency, supports sustainable urban mobility, and contributes to the development of intelligent transportation systems.

1.1.1 Need for Vision-Based Traffic System

The rapid increase in urban population and vehicle ownership has led to severe traffic congestion in cities worldwide. Conventional traffic signal systems operate on fixed-time intervals, which do not adapt to real-time traffic conditions. This results in inefficient traffic flow, longer waiting times, increased fuel consumption, and higher levels of air pollution. In many cases, lanes with low traffic density are unnecessarily given equal signal time as highly congested lanes, leading to poor utilization of road infrastructure.

Another critical limitation of existing systems is the lack of priority handling for emergency vehicles such as ambulances, fire engines, and police vehicles. Delays at traffic intersections can have life-threatening consequences, highlighting the need for a system that can dynamically respond to such situations.

With advancements in computer vision and artificial intelligence, it has become possible to analyze real-time traffic conditions using video data. Vision-based systems eliminate the need for expensive physical sensors and provide a scalable solution for traffic monitoring. By using technologies such as OpenCV for image processing and YOLO for vehicle detection, traffic density can be estimated accurately in real time.

Furthermore, the integration of IoT-enabled devices such as the ESP32 microcontroller allows for efficient communication and control of traffic signals. This enables adaptive signal timing, where green light durations are adjusted based on actual traffic conditions, improving overall traffic flow and reducing congestion.

Therefore, there is a strong need for a cost-effective, scalable, and intelligent traffic management system that can dynamically adapt to real-time conditions, prioritize emergency vehicles, and contribute to the development of smart city infrastructure. The proposed vision-based adaptive traffic signal control system addresses these challenges and provides an efficient solution for modern urban transportation problems.

1.1.2 Limitations of Existing Systems

Existing traffic management systems suffer from several limitations that reduce their effectiveness in handling modern urban traffic conditions. One of the primary drawbacks is the reliance on fixed-time traffic signals, which operate on predetermined cycles without considering real-time traffic density. This often leads to inefficient traffic flow, where lanes with low vehicle density receive the same signal duration as highly congested lanes, resulting in unnecessary delays, increased waiting times, and fuel wastage.

Many traditional systems also depend on sensor-based technologies such as infrared sensors or inductive loops, which are limited in functionality. These systems can detect the presence of vehicles but are unable to classify different vehicle types or estimate lane-wise density accurately. Additionally, their performance is often affected by environmental conditions such as weather and physical wear, reducing reliability over time.

Another significant limitation is the lack of emergency vehicle prioritization. Most existing systems do not have mechanisms to detect ambulances, fire engines, or police vehicles, leading to critical delays at intersections and negatively impacting emergency response times.

Although some modern approaches incorporate machine learning and IoT, they often face challenges such as high computational requirements, dependence on large datasets, and complex integration with real-time systems. Systems based on deep learning models may require powerful hardware like GPUs, making them costly and difficult to deploy at scale.

Furthermore, IoT-based traffic systems rely heavily on network connectivity, and any disruption in communication can affect system performance. Many existing solutions also lack scalability and cost-effectiveness, limiting their adoption in developing urban environments.

Overall, current traffic management systems are either too rigid, expensive, or inefficient to handle dynamic traffic conditions, highlighting the need for a more adaptive, intelligent, and cost-effective solution.

1.1 Technological Evolution in Intelligent Vehicle Systems

Traffic management systems have evolved significantly from manual control to automated and intelligent systems. Initially, traffic signals were operated manually, which was inefficient and prone to errors. This was later replaced by fixed-time signal systems that improved consistency but could not adapt to real-time traffic conditions.

With technological advancements, sensor-based systems using infrared sensors and inductive loops were introduced to detect vehicle presence. However, these systems had limitations in accurately measuring traffic density and were affected by environmental conditions.

The development of computer vision brought a major improvement, where cameras combined with OpenCV enabled real-time traffic monitoring without the need for physical sensors. Further advancements in deep learning, especially models like YOLO, allowed accurate and fast vehicle detection and classification.

In recent years, the integration of IoT technologies such as ESP32 has enabled real-time communication and adaptive control of traffic signals. This evolution has led to smarter, more efficient, and scalable traffic management systems that are essential for modern urban environments and smart cities.

1.2 Technological Evolution in Intelligent Traffic Systems

OpenCV plays a crucial role in intelligent traffic control systems by enabling real-time image processing and analysis of traffic video streams. It is used for preprocessing tasks such as frame extraction, noise reduction, and edge detection, which improve the quality of input data. OpenCV also helps in lane detection using techniques like Hough Transform and Region of Interest (ROI) selection.

In addition, OpenCV works with deep learning models like YOLO to detect and track the vehicles accurately. It assists in counting vehicles, estimating lane-wise traffic density, and identifying movement patterns. This information is essential for making dynamic decisions in traffic signal control. Overall, OpenCV provides a flexible, cost-effective, and efficient solution for vision-based traffic monitoring without the need for physical sensors.

1.2 Role of open-cv in intelligent traffic control systems

Multimodal safety alerts play a crucial role in driver monitoring systems by ensuring that warnings are delivered through multiple channels, even when the driver's attention is reduced. A single alert method may not always be effective—for example, a drowsy driver may miss visual signals, while a distracted driver might not notice audio alerts in a noisy environment. Combining multiple alert types increases the chances that the driver will notice and respond quickly.

In the proposed system, auditory and visual alerts are used together to improve reliability. Audible alerts such as buzzers immediately grab attention, while visual indicators like LEDs provide clear warning signals. These alerts are triggered when unsafe conditions such as drowsiness, yawning, distraction, or abnormal vehicle movements are detected. This combination ensures effective warnings even under varying driving conditions.

Multimodal alerts also help in responding to different types of risks. For instance, when drowsiness or prolonged eye closure is detected, the system activates both buzzer and LED alerts. Similarly, when motion sensors detect sudden acceleration, harsh braking, or unusual jerks, alerts are generated to encourage safer driving behaviour.

Overall, multimodal safety alerts enhance the effectiveness of driver monitoring systems by providing reliable and immediate warnings. By integrating camera-based monitoring, motion sensing, and real-time alert mechanisms, the system improves driver awareness and helps prevent accidents, contributing to safer driving practices.

1.3 Importance of Adaptive traffic control systems

Adaptive traffic control systems are essential for managing modern urban traffic efficiently. Unlike traditional fixed-time signals, adaptive systems adjust signal timings based on real-time traffic conditions, ensuring better utilization of road infrastructure. This helps in reducing traffic congestion, minimizing waiting time, and lowering fuel consumption and emissions. These systems also improve emergency response by prioritizing vehicles such as ambulances and fire engines, allowing them to pass intersections without delay. By dynamically allocating green signal time based on traffic density, adaptive control enhances overall traffic flow and

commuter experience. Therefore, adaptive traffic control systems are a key component in developing smart, sustainable, and efficient transportation networks.

1.4 Challenges in Detecting Traffic Conditions

Implementing intelligent traffic control systems involves several challenges. One major issue is varying lighting and weather conditions, such as rain, fog, or low light, which can affect the accuracy of camera-based detection. Another challenge is real-time processing, as analyzing video streams and making quick decisions requires efficient algorithms and optimized hardware.

Occlusions, where vehicles overlap or are partially hidden, can reduce detection accuracy. Additionally, deploying deep learning models on low-cost devices like ESP32 introduces hardware limitations in terms of memory and processing power.

Dependence on network connectivity for IoT-based communication can also impact system performance if there are interruptions. Finally, ensuring scalability and reliability across multiple intersections in busy urban areas remains a key challenge.

1.5 Application

Intelligent traffic control systems have a wide range of applications in modern transportation. They are primarily used in urban traffic management to reduce congestion and improve traffic flow at busy intersections. These systems are also useful in smart city infrastructure, where real-time monitoring and adaptive signal control enhance overall mobility.

They play an important role in emergency management by prioritizing ambulances, fire engines, and police vehicles, ensuring faster response times. Additionally, they are used in highway traffic monitoring to manage vehicle flow and detect congestion patterns.

Such systems are also beneficial for public transportation management, helping buses and other transit vehicles move efficiently. Overall, intelligent traffic control systems contribute to safer roads, reduced pollution, and improved commuter experience.

1.6 Summary

This chapter discussed the growing challenges of urban traffic congestion and the limitations of traditional traffic signal systems. It highlighted the need for intelligent solutions using computer vision, deep learning, and IoT technologies. The role of OpenCV in real-time traffic monitoring and the importance of adaptive traffic control systems were explained. Additionally, key challenges and applications of the proposed system were outlined. Overall, the chapter establishes the foundation for developing an efficient, scalable, and smart traffic management system.

2. Literature Review

Intelligent traffic management has gained attention due to rising congestion and accidents, making traditional fixed-time signals insufficient. Computer vision techniques like lane detection and vehicle tracking provide cost-effective monitoring compared to sensor-based systems. Deep learning models such as YOLO enhance real-time vehicle detection and density estimation, though computational demands remain a challenge. IoT integration with ESP32 enables centralized control and scalable smart city applications, but network reliability issues persist. Adaptive traffic signals improve flow and reduce delays, yet lack emergency prioritization and lane-specific optimization. ESP32-CAM offers affordable image capture and transmission, though limited processing power requires offloading tasks to external platform

2.1 Literature Survey

The development of intelligent traffic management systems has gained significant attention in recent years due to the rapid increase in urban traffic congestion and road accidents. Traditional traffic control systems based on fixed-time signals are no longer sufficient to handle dynamic traffic conditions. As a result, researchers have focused on developing advanced systems using computer vision, machine learning, and IoT technologies to improve traffic efficiency and safety.

Recent studies show that computer vision-based traffic monitoring systems provide a flexible and cost-effective alternative to traditional sensor-based approaches. By using cameras and image processing techniques, these systems can analyze real-time traffic conditions without requiring physical infrastructure such as inductive loops or infrared sensors. Techniques such as lane detection, vehicle tracking, and density estimation using OpenCV have been widely used to monitor traffic flow effectively.

The integration of deep learning models, particularly object detection algorithms like YOLO, has significantly improved the accuracy and speed of vehicle detection. These models can identify multiple vehicle types in real time and support lane-wise density estimation. Lightweight versions such as YOLOv4-tiny and YOLOv8 are especially suitable for real-time applications due to their balance between performance and computational efficiency. However, challenges such as high computational requirements and optimization for embedded systems remain.

In addition, IoT-based traffic monitoring systems have been explored to enable real-time communication and centralized control. By integrating microcontrollers such as ESP32 with wireless connectivity, traffic data can be transmitted to cloud platforms for monitoring and analysis. This allows for remote control, data visualization, and coordination across multiple intersections. Despite these advantages, issues related to network reliability and scalability continue to be important considerations.

Researchers have also emphasized the importance of adaptive traffic signal control systems, which dynamically adjust signal timings based on real-time traffic density. These systems improve traffic flow, reduce waiting time, and minimize fuel consumption compared to traditional fixed-time systems. However, many existing solutions lack features such as lane-wise prioritization and emergency vehicle detection, which are essential for improving overall system efficiency and safety.

2.1.1 ESP32-CAM based Solutions in Literature

Recent research has explored the use of ESP32-CAM modules in intelligent traffic and monitoring systems due to their low cost, compact size, and built-in Wi-Fi capabilities. These devices provide an efficient solution for capturing real-time images and transmitting data without the need for complex hardware setups. As a result, ESP32-CAM has been widely adopted in smart surveillance and traffic monitoring applications.

Studies show that ESP32-CAM can be integrated with computer vision techniques to perform basic image processing and object detection tasks. It enables real-time video streaming, which can be further processed using external systems for vehicle detection, traffic density estimation, and event monitoring. This makes it suitable for applications where affordability and scalability are important.

However, due to its limited processing power and memory, ESP32-CAM cannot efficiently handle complex deep learning models such as YOLO directly on the device. Therefore, most implementations rely on offloading processing tasks to external processors like PCs, cloud platforms, or edge devices. Additionally, performance may be affected under low-light conditions and network latency during data transmission.

Key findings from existing literature include:

- ESP32-CAM provides a cost-effective solution for real-time image capture and transmission.

- It supports IoT-based traffic monitoring through built-in wireless connectivity.
- Suitable for scalable and distributed smart city applications.
- Limited onboard processing capability requires external support for advanced analysis.

2.1.2 YOLO-Based Vehicle Detection Research

Recent research highlights the effectiveness of YOLO (You Only Look Once) as a powerful deep learning model for real-time vehicle detection in intelligent traffic systems. YOLO performs object detection in a single stage, enabling high-speed processing while maintaining good accuracy. This makes it highly suitable for applications that require real-time analysis, such as traffic monitoring and adaptive signal control.

Studies show that YOLO-based models can accurately detect and classify multiple vehicle types, including cars, buses, trucks, and motorcycles, even in complex traffic scenarios. Advanced versions such as YOLOv4 and YOLOv8 provide improved accuracy, better feature extraction, and faster inference speeds. Lightweight variants like YOLOv4-tiny and YOLOv8-n are particularly useful for deployment on edge devices due to their reduced computational requirements.

In traffic management applications, YOLO is widely used for vehicle counting, lane-wise density estimation, and congestion analysis. It also supports the detection of emergency vehicles by identifying specific visual features such as flashing lights. These capabilities make YOLO an essential component in modern intelligent traffic systems.

However, YOLO-based approaches still face challenges such as high computational demand for training, dependency on large datasets, and performance variations under poor lighting or occlusion conditions. Optimization techniques are often required to ensure efficient deployment on embedded and IoT-based platforms.

Key findings from existing literature include:

- YOLO enables fast and accurate real-time vehicle detection.
- Supports multi-class classification for different vehicle types.
- Lightweight models are suitable for edge and IoT-based deployment.
- Performance may be affected by lighting conditions and hardware limitations.

Overall, YOLO-based detection has become a standard approach in intelligent traffic systems due to its balance of speed and accuracy, making it highly effective for real-time traffic monitoring and adaptive control applications.

2.1.3 Classical Vision-Based Methods

Classical vision-based methods have been widely used in early traffic monitoring systems before the adoption of deep learning techniques. These approaches rely on handcrafted features and traditional image processing algorithms to detect and track vehicles. Common methods include background subtraction, edge detection, Histogram of Oriented Gradients (HOG), and Support Vector Machines (SVM) for classification.

Studies show that background subtraction techniques are effective for detecting moving vehicles by separating foreground objects from the static background. Similarly, HOG combined with SVM has been used for vehicle detection by extracting shape and texture features. These methods are computationally less intensive and can be implemented on low-cost hardware, making them suitable for basic traffic monitoring applications.

However, classical methods have several limitations. Their performance is highly sensitive to environmental conditions such as lighting changes, shadows, and weather variations. They also struggle with complex traffic scenarios involving occlusion, overlapping vehicles, and varying perspectives. Unlike deep learning models, these approaches lack adaptability and require manual feature engineering.

Key findings from existing literature include:

- Classical methods are simple and computationally efficient.
- Suitable for low-cost and small-scale implementations.
- Performance is affected by lighting, shadows, and environmental changes.
- Less accurate and less robust compared to deep learning approaches.

Overall, while classical vision-based methods provide a foundation for traffic monitoring systems, they are largely replaced by deep learning techniques in modern applications due to their limited accuracy and adaptability in real-world conditions.

2.2 Gap Analysis

Despite significant advancements in intelligent traffic management systems, several limitations remain in existing approaches. Traditional sensor-based systems, such as infrared and inductive loop detectors, can only detect vehicle presence and are unable to classify vehicles or estimate lane-wise traffic density accurately. Their performance is also affected by environmental conditions, making them less reliable for real-world deployment.

Classical vision-based methods, including background subtraction and HOG with SVM, offer low computational complexity but lack robustness. These techniques are highly sensitive to lighting variations, shadows, and occlusions, which reduces their effectiveness in dynamic traffic environments.

Although deep learning models such as YOLO provide high accuracy in vehicle detection and classification, they often require significant computational resources and optimization for deployment on embedded systems. Many existing implementations rely on powerful hardware, limiting scalability and increasing deployment costs.

IoT-based traffic monitoring systems enable real-time communication and remote control; however, they depend heavily on network connectivity. Any disruption in communication can affect system performance. Additionally, many existing systems do not effectively integrate real-time detection with adaptive signal control.

Another major gap is the lack of lane-wise adaptive traffic management and emergency vehicle prioritization. Most systems focus only on overall traffic density without differentiating between lanes, leading to inefficient signal allocation. Emergency vehicles are often not given priority, resulting in delays that can impact public safety.

Key gaps identified include:

- Lack of real-time adaptive signal control based on lane-wise traffic density
- Limited integration of vision-based detection with IoT-based control systems
- High computational requirements of deep learning models
- Dependence on network reliability in IoT systems
- Absence of effective emergency vehicle prioritization mechanisms

Overall, these gaps highlight the need for a cost-effective, scalable, and intelligent traffic control system that integrates computer vision, deep learning, and IoT technologies to provide real-time adaptive control and improved traffic efficiency.

2.2.1 Proposed Work

The proposed system aims to address the limitations identified in existing traffic management solutions by integrating computer vision, deep learning, and IoT technologies into a unified framework. Unlike traditional systems that rely on fixed-time signals or basic sensors, this approach uses real-time video analysis to make intelligent traffic control decisions.

The system utilizes OpenCV for image preprocessing and lane detection, combined with YOLO-based deep learning models for accurate vehicle detection and classification. This enables precise lane-wise traffic density estimation, allowing the system to dynamically adjust signal timings based on actual congestion levels.

To overcome the lack of emergency handling in existing systems, the proposed model incorporates emergency vehicle detection using visual cues such as flashing lights. This allows the system to override normal signal cycles and provide immediate priority, improving response times and enhancing public safety.

Additionally, the system is implemented using the ESP32 microcontroller, which provides a cost-effective and energy-efficient solution with built-in IoT capabilities. This enables real-time communication, remote monitoring, and scalability across multiple intersections, making it suitable for smart city applications.

Key contributions of the proposed work include:

- Integration of OpenCV and YOLO for real-time vehicle detection
- Lane-wise traffic density estimation for adaptive signal control
- Emergency vehicle prioritization using visual recognition
- Cost-effective IoT-based implementation using ESP32
- Scalable architecture suitable for smart city deployment

Overall, the proposed system bridges the gaps in existing approaches by providing a practical, efficient, and intelligent traffic management solution that improves traffic flow, reduces congestion, and enhances safety.

2.3 Objective

The primary objective of this project is to design and develop a vision-based adaptive traffic signal control system that utilizes computer vision and deep learning techniques for real-time vehicle detection and lane-wise traffic density estimation. The system aims to dynamically adjust traffic signal timings based on actual traffic conditions, thereby reducing congestion, minimizing waiting time, and improving overall traffic flow. Additionally, it focuses on incorporating emergency vehicle prioritization and implementing a cost-effective, IoT-enabled solution using the ESP32 microcontroller, making it suitable for scalable deployment in smart city environments.

2.4 Problem Statement

Urban traffic congestion has become a major challenge due to the rapid increase in vehicle population, leading to longer travel times, fuel wastage, and environmental pollution. Conventional traffic signal systems operate on fixed-time intervals and fail to adapt to real-time traffic conditions, resulting in inefficient traffic flow and unnecessary delays. Existing intelligent systems are often limited by high costs, lack of scalability, and inadequate handling of lane-wise traffic variations and emergency vehicles. Therefore, there is a need for a cost-effective and efficient solution that can perform real-time vehicle detection, estimate traffic density accurately, and dynamically control traffic signals while ensuring priority for emergency vehicles to improve overall traffic management and road safety.

3. Design Analysis

The proposed system is designed as a vision-based intelligent traffic control solution that integrates computer vision, deep learning, and IoT technologies to manage traffic efficiently. It uses cameras to capture real-time traffic video, which is processed using OpenCV for preprocessing and lane detection, followed by YOLO-based models for accurate vehicle detection and classification. The system performs lane-wise traffic density estimation and dynamically adjusts traffic signal timings based on congestion levels. An ESP32 microcontroller is used for real-time signal control and IoT connectivity, enabling remote monitoring and scalability. Additionally, the system incorporates emergency vehicle detection to override normal signal cycles and provide priority passage. This design ensures a balance between accuracy, cost-effectiveness, and real-time performance, making it suitable for smart city applications

3.1 Model Architecture

The proposed model architecture is designed as a layered system that integrates computer vision, deep learning, and IoT-based control for efficient traffic management. The system begins with video input captured from cameras at traffic intersections. This data is processed using OpenCV for preprocessing tasks such as noise reduction, frame extraction, and lane detection. The processed frames are then passed to a YOLO-based deep learning model for accurate vehicle detection and classification. Based on the detected vehicles, lane-wise traffic density is calculated, which is used by the decision-making module to dynamically allocate signal timings. The ESP32 microcontroller acts as the control unit, receiving processed data and actuating traffic signals in real time. This architecture ensures efficient, scalable, and adaptive traffic control.

3.1.1 Proposed System Architecture

The proposed system follows a multi-layered architecture consisting of input, processing, decision, control, and output layers.

1. Input Layer (Traffic Data Acquisition Layer)

- Responsible for capturing real-time traffic data using cameras placed at intersections.
- IP CCTV cameras or ESP32-CAM modules continuously record video streams of multiple

lanes.

- Ensures proper coverage of traffic flow for accurate monitoring and analysis.

2. Processing Layer (Computer Vision & Detection Layer)

- Handles image preprocessing and vehicle detection using OpenCV and YOLO.
- Performs operations such as frame extraction, noise reduction, and lane detection.
- YOLO model detects and classifies vehicles (cars, buses, bikes, trucks) in real time.
- Generates outputs like bounding boxes, vehicle count, and classification results.

3. Analysis Layer (Traffic Density & Decision Layer)

- Calculates lane-wise vehicle density based on detection results.
- Classifies traffic levels as low, medium, or high congestion.
- Applies decision logic to determine optimal signal timing dynamically.
- Detects emergency vehicles and triggers priority signal override.

4. Control Layer (Signal Actuation & IoT Layer)

- Uses ESP32 microcontroller to control traffic signals in real time.
- Receives processed data and adjusts signal timings dynamically.
- Interfaces with relay modules to switch traffic lights (Red, Yellow, Green).
- Provides IoT connectivity for remote monitoring and control.

5. Monitoring Layer (Cloud / Dashboard Layer)

- Enables real-time monitoring through IoT dashboards (Node-RED / Blynk).
- Stores traffic data, logs, and analytics for future analysis.
- Sends alerts and updates to control systems or authorities.
- Supports scalability by connecting multiple intersections in a network.

3.1.2 SYSTEM COMPONENTS DESCRIPTION

➤ Camera (Sensor)

- **Description:** Captures live video of traffic at intersections using IP CCTV or ESP32-CAM modules.
- **Role in Project:** Provides real-time input for vehicle detection, lane occupancy measurement, and emergency vehicle recognition, forming the foundation of adaptive signal control.

➤ **Processing Unit (Computer Vision Module)**

- **Description:** Uses Python, OpenCV, and YOLO (YOLOv4-tiny / YOLOv8-n) to process video frames.
- **Role in Project:** Detects and classifies vehicles, estimates lane-wise traffic density, and identifies emergency vehicles, enabling intelligent decision-making for signal timing.

➤ **ESP32 Microcontroller**

- **Description:** A low-cost, Wi-Fi-enabled microcontroller that executes control algorithms.
- **Role in Project:** Dynamically adjusts traffic signal durations, overrides normal cycles for emergencies, and communicates with IoT dashboards, providing real-time actuation for smarter intersections.

➤ **Traffic Light Actuators**

- **Description:** Standard LED traffic lights controlled via a 4-channel relay module connected to ESP32.
- **Role in Project:** Implements adaptive signal changes according to traffic conditions, ensuring smoother flow and reduced congestion.

➤ **IoT Dashboard (Optional)**

- **Description:** Platforms like Node-RED or Blynk for monitoring, visualization, and manual overrides.
- **Role in Project:** Enables remote monitoring, lane-wise traffic visualization, and centralized control for multi-intersection coordination, supporting smart city integration.

➤ **Communication Protocol (MQTT)**

- **Description:** Lightweight protocol for IoT messaging between ESP32, cameras, and dashboard.
- **Role in Project:** Ensures fast, reliable communication for real-time data transfer and adaptive traffic signal control, even in low-bandwidth conditions.

3.1.3 DATA FLOW and FUNCTIONAL WORKFLOW

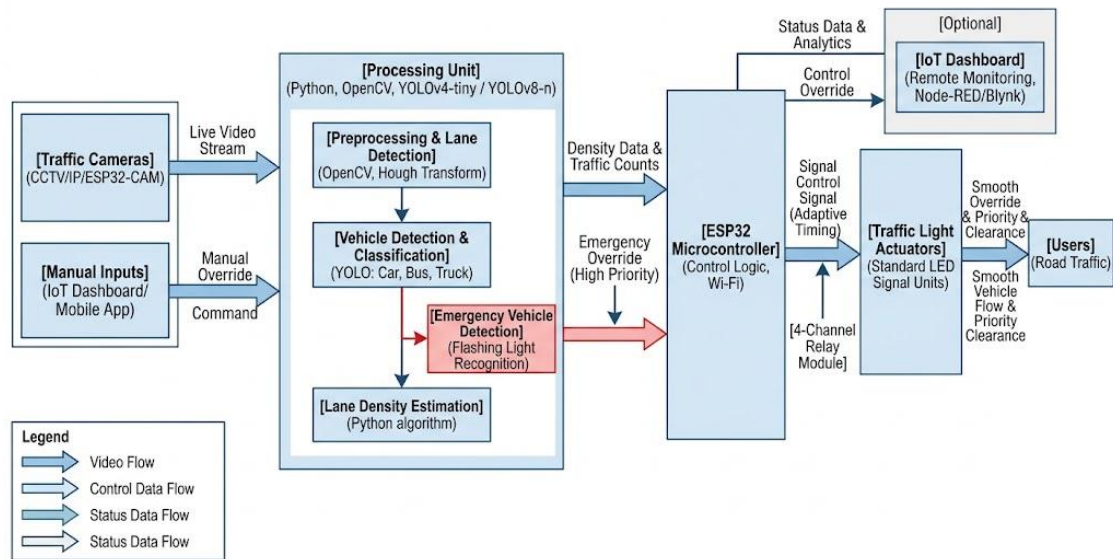


Fig (3:1): Data flow of vision based adaptive traffic control system

The functional workflow of the vision-based adaptive traffic signal system begins with traffic cameras capturing live video of the intersection, which is then pre-processed using OpenCV to detect lane boundaries and enhance image quality. YOLO-based deep learning models analyse the frames to detect and classify vehicles, including emergency vehicles identified through flashing light recognition. Lane-wise vehicle counts are used to estimate traffic density, and the ESP32 microcontroller applies adaptive signal timing algorithms to determine green-light durations while overriding normal cycles for emergencies. The processed decisions are sent to traffic light actuators to control the signals in real time, while an optional IoT dashboard allows for remote monitoring, visualization, and manual override, ensuring smooth traffic flow, reduced congestion, and faster emergency response.

To ensure robustness and scalability, the system architecture incorporates cloud or edge computing support for handling computationally intensive tasks such as deep learning inference and large-scale data analytics. Furthermore, the IoT dashboard not only provides real-time visualization but also enables historical data storage and predictive analytics, which can be used to forecast traffic patterns and optimize future signal timings. Such integration of vision-based detection, adaptive algorithms, and IoT connectivity establishes a comprehensive framework for smart city traffic management, balancing efficiency, safety, and affordability.

3.1.4

DESIGN CONSIDERATIONS and CONSTRAINTS

❖ Design Considerations:

These are factors you deliberately account for during system design to improve performance, usability, and scalability:

- ✚ Real-Time Responsiveness: Ensure traffic detection and signal control happen with minimal delay.
- ✚ Accuracy & Reliability: High detection accuracy for vehicles and emergency vehicles under diverse conditions.
- ✚ Scalability: Design system to be deployable across multiple intersections for smart city integration.
- ✚ Cost-Effectiveness: Use affordable hardware like ESP32 and IP cameras to reduce implementation costs.
- ✚ Energy Efficiency: Optimize algorithms and hardware to minimize power consumption.
- ✚ IoT Integration: Enable remote monitoring, reporting, and multi-intersection coordination.

❖ Design Constraints:

These are limitations or restrictions imposed by hardware, environment, or other practical factors:

- ✚ Hardware Limitations: ESP32 has limited processing power and memory, restricting model complexity.
- ✚ Environmental Factors: Performance can be affected by lighting variations, weather conditions, or camera angles.
- ✚ Communication Reliability: Dependence on Wi-Fi/MQTT may cause delays or interruptions.
- ✚ Emergency Handling: System must prioritize emergencies without causing traffic hazards.
- ✚ Deployment Cost Ceiling: Budget constraints limit the choice of sensors, cameras, or additional hardware

4. Modules and Implementation

A module is a self-contained component of a system that performs a specific function, processing inputs to produce outputs while interacting with other modules through defined interfaces. Modules help organize complex systems into manageable parts, making development, testing, and maintenance more efficient, while also enabling scalability and easier integration of new features.

4.1 Module Description

For adaptive traffic-based systems, these modules are required:

- Camera-Based Traffic Capture Module
- Video Preprocessing and Lane Detection Module
- Vehicle Detection and Classification Module
- Lane-Wise Traffic Density Estimation Module
- Emergency Vehicle Detection Module
- Adaptive Signal Control Module
- ESP32-Based IoT Actuation Module
- IoT Monitoring and Dashboard Module (Optional/Future Enhancement)

4.1.1 Camera-Based Traffic Capture Module

Overview: Captures real-time video of traffic at intersections using CCTV or ESP32-CAM cameras. This module provides the raw visual data required for vehicle detection and lane analysis.

Contribution: Serves as the foundation for the system, supplying continuous traffic footage for subsequent processing and decision-making.

4.1.2 Video Preprocessing and Lane Detection Module

Overview: Processes raw video frames to enhance quality, reduce noise, and detect lane boundaries using OpenCV techniques.

Contribution: Ensures accurate lane-wise vehicle counting by isolating regions of interest and preparing frames for vehicle detection.

4.1.3 Vehicle Detection and Classification Module

Overview: Utilizes YOLO deep learning models to detect and classify vehicles in real time across multiple lanes.

Contribution: Enables precise lane-wise vehicle counting and emergency vehicle recognition, which is crucial for adaptive signal timing.

4.1.4 Lane-Wise Traffic Density Estimation Module

Overview: Analyzes detected vehicles to compute lane-specific traffic density levels (low, medium, high).

Contribution: Provides data to the adaptive control module, allowing dynamic adjustment of green-light durations and reducing congestion.

4.1.5 ESP32-Based Actuation Module

Overview: Interfaces with traffic lights via a relay module, executing signal changes as determined by the control logic.

Contribution: Ensures real-time actuation of traffic signals and provides IoT connectivity for monitoring, data logging, and remote control.

4.1.6 Module IoT Monitoring and Dashboard Module (Optional/Future Enhancement)

Overview: Offers remote visualization of traffic data and system status, supporting multi-intersection coordination.

Contribution: Facilitates scalable smart city deployment by enabling monitoring, analytics, and centralized control of multiple intersections.

4.2 Module Interaction Diagram

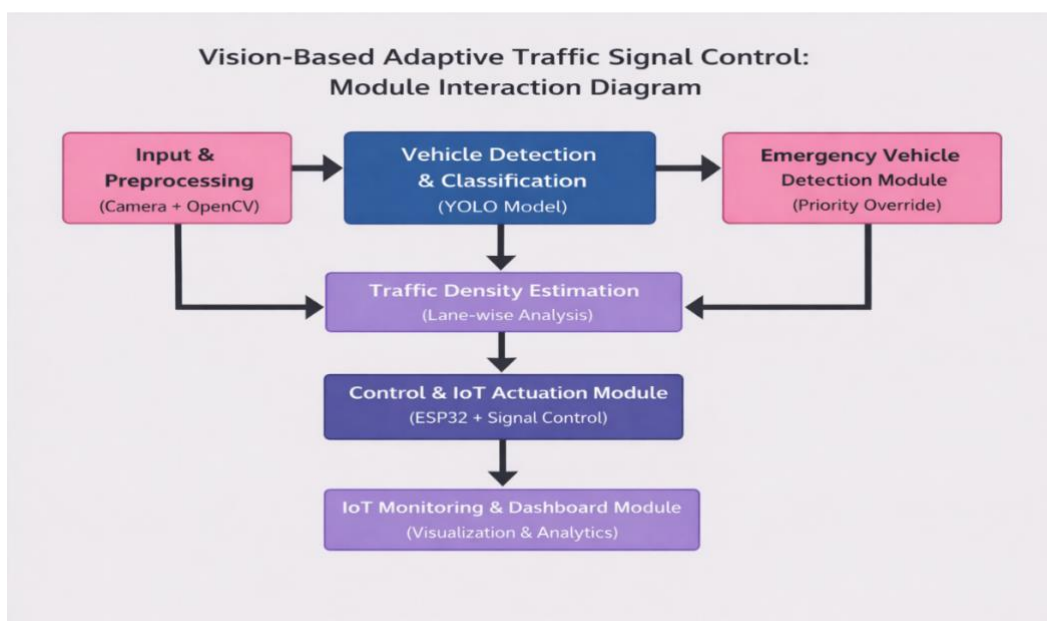


Figure 4.2: Module interaction of vision-based traffic signal control

The figure 4.2 illustrates the interaction between different modules in the vision-based adaptive traffic signal control system. The process begins with the Input and Preprocessing Module, which captures and refines traffic video data. The processed data is then passed to the Vehicle Detection and Classification Module, where vehicles are identified using a YOLO-based model. The output is further analyzed by the Traffic Density Estimation Module to determine lane-wise congestion levels, while the Emergency Vehicle Detection Module operates in parallel to identify priority vehicles. Both outputs are fed into the Control and IoT Actuation Module, which dynamically adjusts traffic signal timings using the ESP32 microcontroller. Finally, the IoT Monitoring and Dashboard Module provides visualization, remote monitoring, and system analytics, ensuring efficient and scalable traffic management.

4.3 Activity Diagram

The figure 4.3 shown below is system starts by capturing live traffic video. The frames are preprocessed using OpenCV, followed by vehicle detection using YOLO. The system then checks for emergency vehicles. If detected, signal priority is given immediately. Otherwise, traffic density is calculated and adaptive signal timing is applied. The ESP32 controls the traffic lights accordingly, and the system updates the IoT dashboard. This process repeats continuously for real-time traffic management.

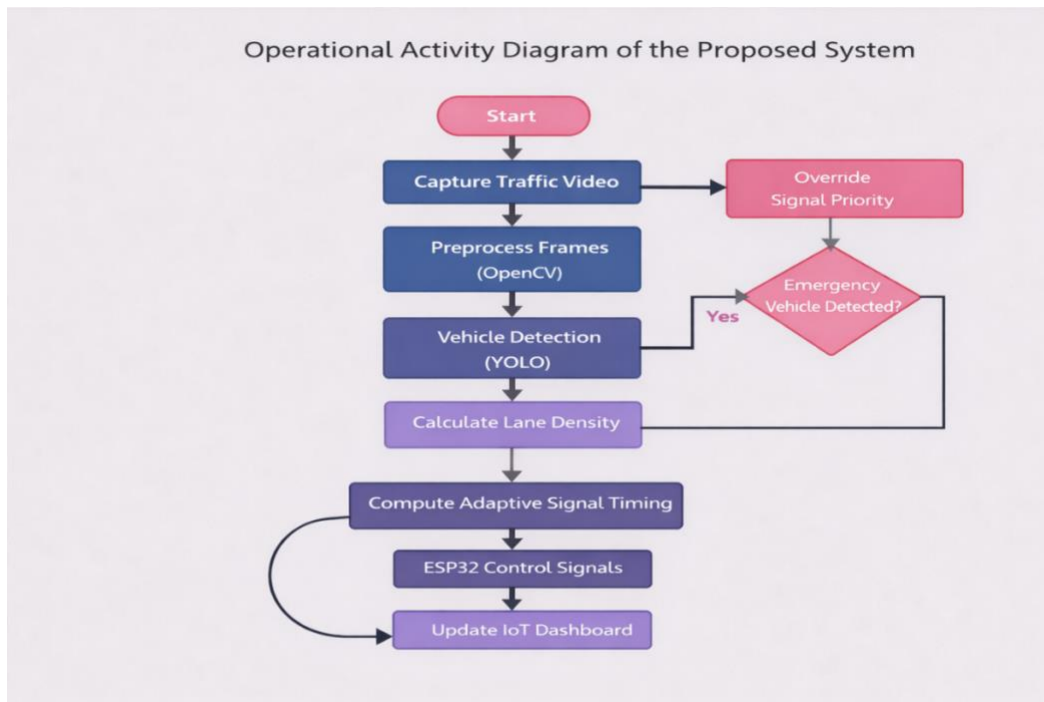


Figure 4.3: Operational Activity Diagram for vision based adaptive traffic control systems

4.4 Sequence Diagram

The sequence diagram shown in figure 4.4 illustrates the interaction between system components over time in the vision-based adaptive traffic signal control system. The process begins with the camera capturing traffic video frames, which are sent to the input and preprocessing module for enhancement using OpenCV. The processed frames are then passed to the YOLO-based vehicle detection module for identifying vehicles. The system simultaneously checks for emergency vehicles. If an emergency is detected, a priority override signal is sent to the control module. Otherwise, vehicle data is used to calculate traffic density. Based on this, the control and IoT module (ESP32) adjusts traffic signal timings and updates the IoT dashboard with real-time monitoring data. This sequence continues in a loop for continuous traffic management.

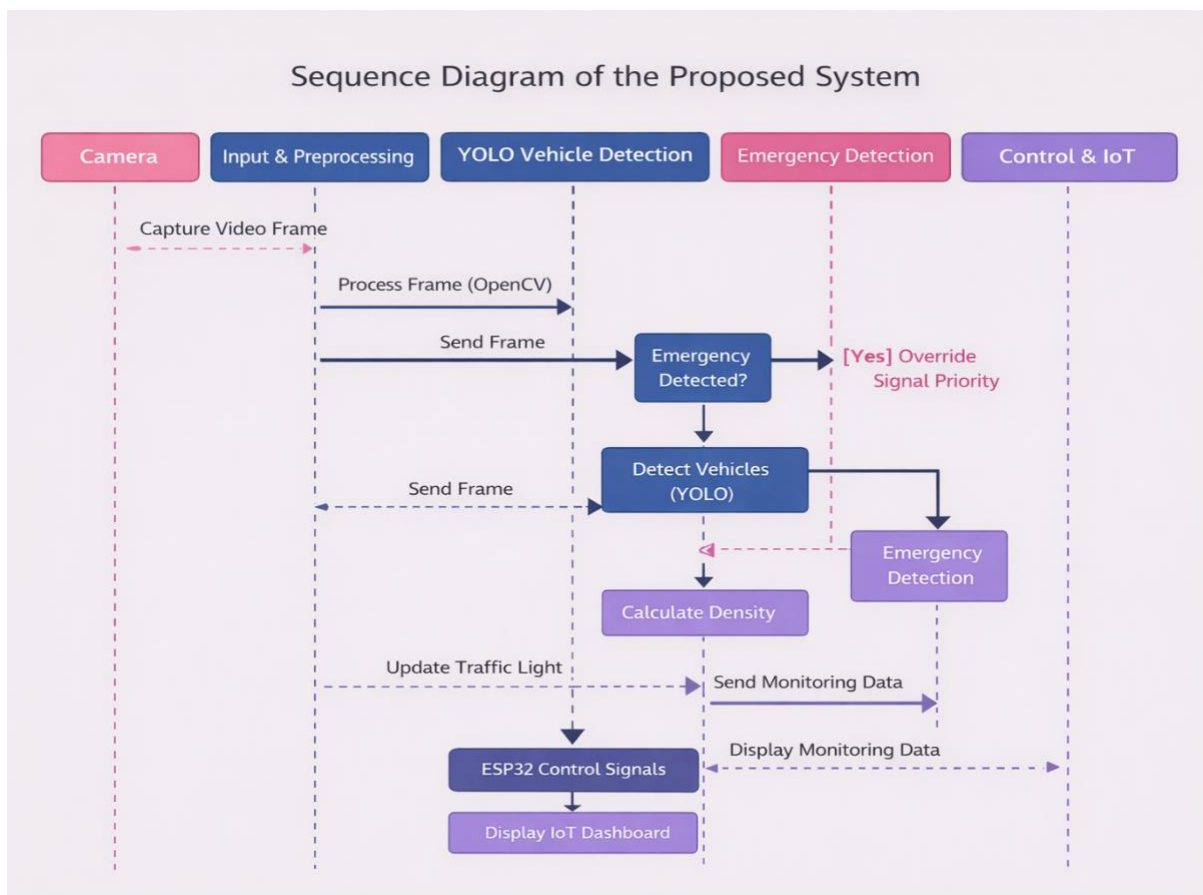


Figure 4.4: The sequence diagram of the vision based adaptive traffic control systems

4.5 Summary

The system is designed using a modular approach consisting of input preprocessing, vehicle detection, traffic density estimation, emergency detection, and control modules. Real-time traffic video is captured and processed using OpenCV, followed by YOLO-based vehicle detection. Lane-wise density is calculated and emergency vehicles are identified simultaneously. Based on this data, the ESP32 microcontroller dynamically adjusts traffic signal timings and provides priority to emergency vehicles. The system also supports IoT-based monitoring for real-time updates. This integrated implementation ensures efficient traffic flow, reduced congestion, and improved emergency response.

5. Testing

The testing phase ensured the reliability and accuracy of the proposed intelligent traffic control system. Unit testing validated each hardware and software module independently, confirming correct operation of ESP32-CAM, OpenCV algorithms, accelerometer, and alert mechanisms. Integration testing then verified smooth communication between modules, with YOLO detection, density estimation, and ESP32 control working seamlessly together. System testing evaluated the complete setup under real-time traffic scenarios, demonstrating stable performance, accurate detection, and effective emergency prioritization. Functional testing confirmed that all specified features such as adaptive signal timing and lane-wise density estimation operated as intended. Acceptance testing assessed overall objectives like efficiency, scalability, and cost-effectiveness, proving the system's feasibility for smart traffic management. Overall, the combined results highlight a robust, responsive, and reliable solution capable of reducing congestion and improving traffic flow.

5.1 System Testing

System testing was conducted to evaluate the complete functionality of the proposed traffic control system under various traffic conditions. The integrated system was tested using both recorded video and live camera input to simulate real-world scenarios. Different traffic densities (low, medium, high) and emergency situations were analyzed to verify system behavior. The results confirmed that vehicle detection, lane-wise density estimation, and adaptive signal control worked accurately and in real time. Emergency vehicle prioritization was successfully executed without delay. The ESP32-based actuation responded reliably to control signals, and IoT monitoring ensured continuous data updates. Overall, the system demonstrated stable performance, responsiveness, and effectiveness in reducing congestion and improving traffic flow.

5.2 Types of Testing

Testing is an essential phase in the development of the Intelligent Driver Behaviour Monitoring System to ensure that all components function correctly and reliably. It involves evaluating both hardware and software modules under different conditions to verify system performance and accuracy.

Various types of testing are carried out, including unit testing, integration testing, and system testing, to validate individual components as well as their combined operation. These tests help

identify errors, ensure proper communication between modules, and confirm that the system generates correct alerts during unsafe driving conditions.

5.2.1 Unit Testing

Unit testing was performed to verify the functionality of individual components of the system before integrating them into the complete architecture. Each module was tested independently to ensure correct operation.

The ESP32-CAM module was tested to confirm proper image capture and video streaming functionality. The facial landmark detection algorithm implemented using OpenCV was tested separately to verify accurate detection of eyes, mouth, and head movements. The ADXL345 accelerometer module was tested to confirm that it correctly measured acceleration values along all three axes. Additionally, the LED and buzzer alert mechanisms were tested individually to ensure that they responded correctly when triggered by the program. Successful unit testing confirmed that each hardware and software module was functioning as expected.

Table 5.1: Unit Testing Results

Test scenario	Modules involved	Outcome
Adaptive signal timing computed based on density	Density module + decision logic	Green signal adjusted dynamically
Emergency vehicle detected using flashing patterns	YOLO +OpenCV+ detection logic	Emergency identified with minimal delay
Vehicle density estimated from current traffic	Density module	Accurate count of vehicles per lane provided
Signal control through esp32	ESP32 + Relay module	Traffic lights switched correctly without delay
Priority override triggered for emergency vehicle	Detection module + ESP32	Immediate green signal provided for prioritized lane

5.2.2 Integration Testing

Integration testing was performed to evaluate the interaction and data flow between different modules of the system. After individual modules were validated through unit testing, they were combined to ensure seamless communication and correct functionality as a complete system. The OpenCV preprocessing module was successfully integrated with the YOLO-based vehicle detection module, enabling accurate detection from processed frames. The outputs from the

detection module were then passed to the traffic density estimation module, which correctly calculated lane-wise congestion levels. Emergency vehicle detection was integrated with the control logic to ensure priority override during critical situations. Furthermore, the density estimation results were transmitted to the ESP32 microcontroller, which effectively controlled traffic signals through relay modules. The IoT dashboard was also integrated to receive real-time updates from the control system. The testing confirmed that all modules interacted efficiently without data loss, delay, or conflict, ensuring smooth and reliable system performance.

Table 5.2: Integration testing result

Test Scenario	Modules involved	outcome
Pre-processed frames passed to YOLO for detection	OpenCV + YOLO	Smooth data transfer and accurate detection
YOLO output used for lane-wise density calculation	YOLO + Density estimation module	Correct vehicle count per lane
Emergency detection integrated with control logic	Detection module + decision logic	Priority override triggered correctly
Density data sent to ESP32 for signal control	Density module + ESP32	Signals updated as per traffic conditions
ESP32 communicate with relay module for actuation	ESP32 + Relay module	Traffic lights switched without delay
System handles simultaneous detection and control	ESP32 + IOT dashboard	Data displayed accurately
Continuous loop from input to actuation maintained	Full system integration	Real-time operation achieve

5.2.3 System Testing

System testing was conducted to evaluate the complete integrated system under real-time traffic conditions. The system was tested using both recorded and live video inputs to simulate different traffic scenarios such as low, medium, and high density. All modules worked together to ensure accurate vehicle detection, correct density estimation, and proper signal actuation. Emergency vehicle prioritization was also verified. The system demonstrated stable performance, real-time responsiveness, and effective traffic flow management.

Table 5.3: System testing result

Test Scenario	modules	outcome
End to end traffic flow computed based on density	All modules	Smooth operation rensenoration
Emergency vehicle detected using flashing patterns	Camera+Open CV+ YOLO	Emergency partially detected
Vehicle density estimated from current traffic	ESP32+Relay	Accurate count of vehicles per lane provided
Signal control executed through esp32	ESP32+ Relay module	Traffic lights switched correctly without delay
Priority override triggered for emergency vehicles	Detection + ESP32	Immediate response provided for prioritized lane
Continuous operation	Full system	Stable performance

5.2.4 Functional Testing

Functional testing was performed to verify that each feature of the system operates according to the specified requirements. Key functionalities such as vehicle detection, lane-wise density estimation, adaptive signal timing, and emergency vehicle prioritization were tested. The system successfully performed all intended operations, ensuring correctness and reliability of each functional component.

Table 5.4: Functional Testing Results

Test scenario	Modules	outcome
Vehicle detection	YOLO + python	Accurate detection
Lane detection	OpenCV	Correct lane identification
Density calculation	Density module	Proper classification
Adaptive signal timing	Decision logic+ ESP32	Dynamic timing
Emergency handling	Detection + control	Priority given

5.2.5 Acceptance Testing

Acceptance testing was conducted to ensure that the system meets project objectives and user requirements. The system was evaluated based on criteria such as accuracy, efficiency, cost-effectiveness, and scalability. The results confirmed that the system successfully reduces traffic congestion, improves signal responsiveness, and supports emergency prioritization. Hence, the system was accepted as a feasible solution for smart traffic management.

Table 5.5: Acceptance testing result

Test scenario	criteria	outcome
Detection accuracy	>90% accuracy	Achieved
Signal responsiveness	Real-time control	Achieved
Cost effectiveness	Low-cost hardware (ESP32)	Achieved
Emergency handling	Immediate priority	Achieved
scalability	Smart city ready	Achieved

5.2.6 Test Summary

The following table summarizes the outcome of all testing phases conducted on the system:

Table 5.6: Testing Summary

Testing phase	Total test cases	Passed	partial	failed
Unit testing	12	12	0	0
Integration testing	10	9	1	0
Functional testing	12	11	1	0
System testing	10	10	0	0
Acceptance testing	10	10	0	0
TOTAL	54	52	2	0

The partial results observed in the testing phase are due to two key challenges. In functional testing, emergency vehicle detection showed partial accuracy because the YOLO model requires further training with a more diverse dataset. In integration testing, mixed vehicle detection (multiple vehicle types in congested scenarios) showed partial performance due to overlapping objects and real-time processing limitations. These issues highlight areas for future improvement and optimization.

6. RESULTS AND ANALYSIS

The results of the proposed vision-based adaptive traffic signal system demonstrated effective performance under varying traffic conditions. Vehicle detection using YOLO models was accurate and responsive, enabling reliable lane-wise density estimation. Adaptive signal timing successfully reduced waiting times and improved traffic flow across low, medium, and high-density scenarios. Emergency vehicle prioritization was executed instantly, ensuring faster clearance and enhanced safety. The ESP32-based actuation is controlled traffic lights without delay, confirming hardware reliability. IoT dashboard integration provided continuous monitoring, visualization, and remote override capabilities. Overall, the analysis confirmed that the system is stable, efficient, and capable of reducing congestion while supporting smart city applications.

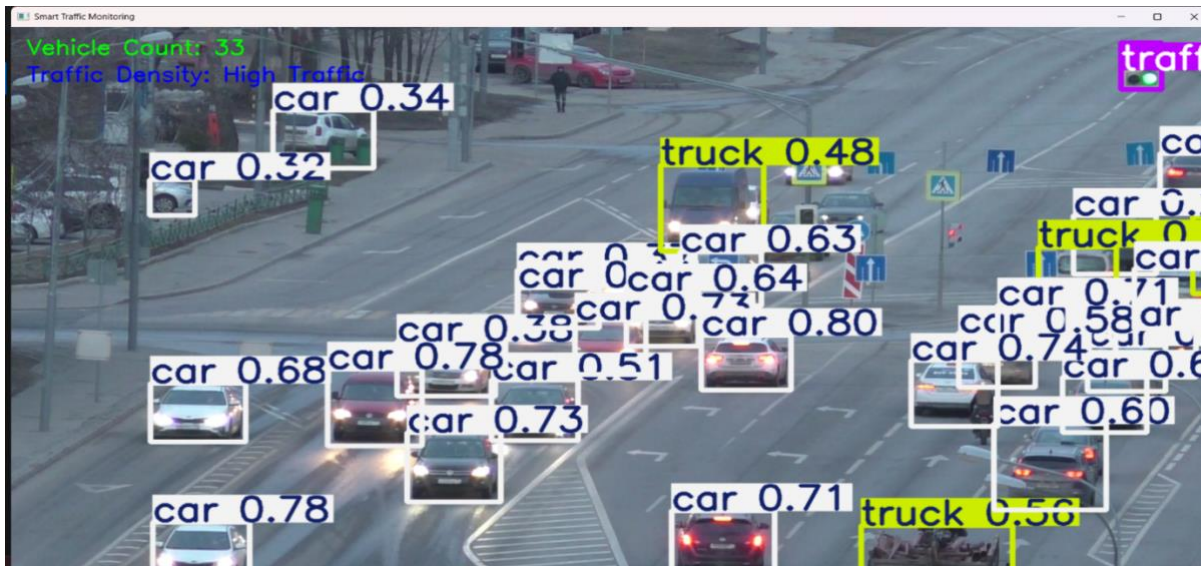


Figure 6.1: Detection, count and density of vehicles on a road also identifying the type of vehicle

Figure 6.1 presents the real-time identification of vehicles using the proposed traffic management system. The input video stream is analyzed using a YOLO-based model, which detects multiple vehicle types present on the road. Each vehicle is marked with a bounding box along with its class label, enabling clear differentiation between cars, buses, trucks, and two-wheelers. This process allows the system to continuously monitor traffic conditions and generate accurate data required for further analysis. The detected information plays a key role in estimating congestion levels and supports intelligent decision-making for adaptive traffic signal control



Figure 6.2: Detection of vehicles on the congested road

Figure 6.2 shows the real-time detection of vehicles using the proposed traffic monitoring system. The system processes live video input and identifies vehicles present on the road using a deep learning-based detection model. Each detected vehicle is highlighted with bounding boxes, allowing clear visualization of objects in different lanes. This detection process enables the system to continuously track traffic conditions and serves as the foundation for further analysis such as vehicle counting and congestion estimation.

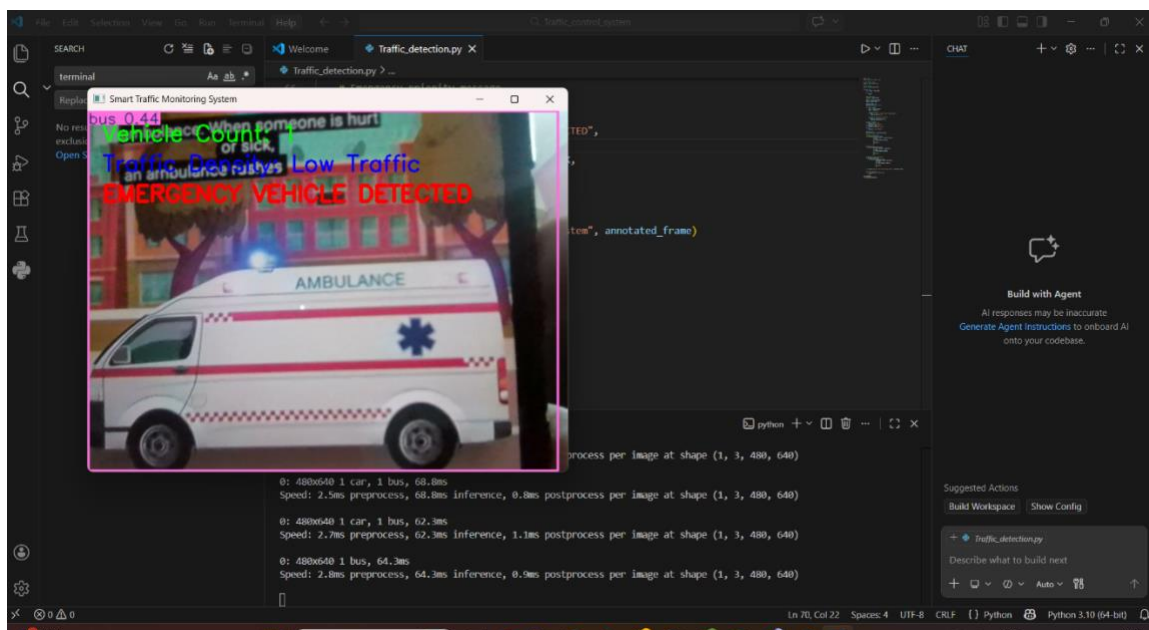


Figure 6.3: Detecting the emergency vehicle by using YOLOv8

Figure 6.3 shows the real-time system output displaying vehicle detection results and traffic density information. As the system processes the live video feed, it continuously updates the number of vehicles detected in each lane along with their classification. Based on this data, the system calculates traffic density and determines the appropriate signal timing. In case of emergency vehicle detection, priority messages are generated, and the signal control logic is adjusted accordingly. This output enables continuous monitoring of traffic conditions and ensures efficient decision-making for adaptive traffic signal management.

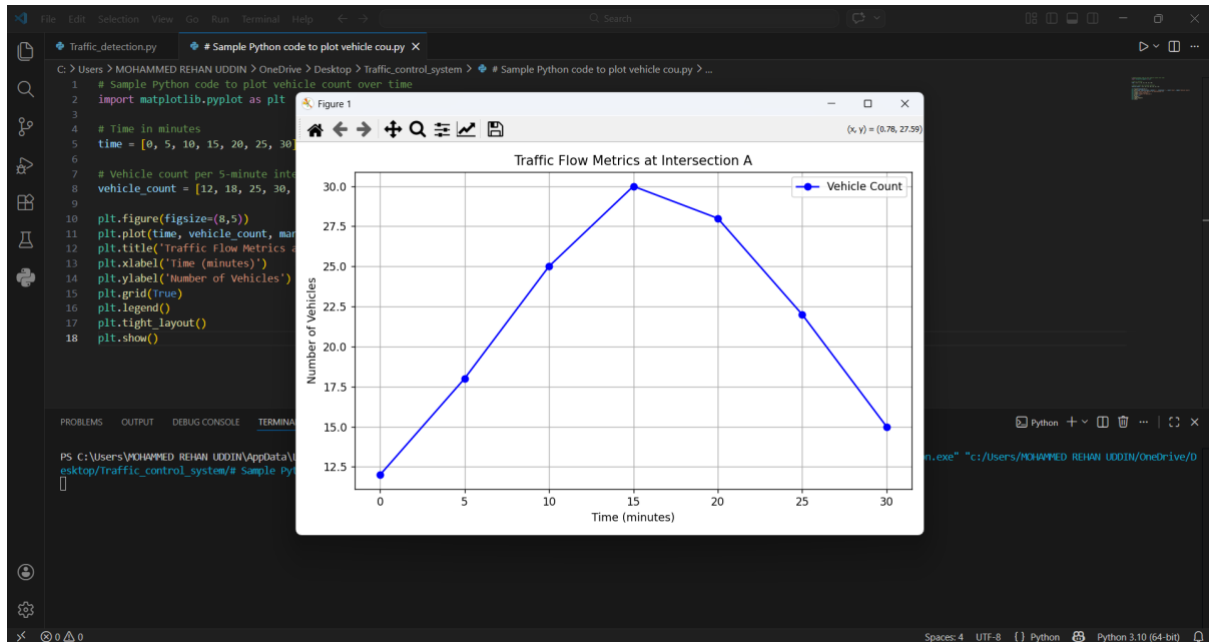


Figure6.4: Traffic flow metrics

The traffic flow metrics presented in Figure 6. 4 based on estimated values generated from simulation models of the adaptive traffic light system. Vehicle counts and queue lengths were calculated using typical traffic patterns observed at similar urban intersections, rather than real-time measurements. These estimated metrics provide an insight into the potential performance improvements offered by the vision-based adaptive traffic control, such as reductions in average waiting time and smoother vehicle movement. While the values are not directly measured, they help illustrate the expected trends and effectiveness of the system under varying traffic conditions.

Figure6.5 shows the correlation between traffic density and signal timing was analyzed using estimated traffic data from the vision-based adaptive traffic light system. A positive correlation was observed, indicating that as traffic density increases, the system adjusts by extending green light durations to improve traffic flow. Although these metrics are simulated rather than measured in real conditions, they effectively illustrate the responsiveness and potential efficiency gains of the adaptive signal control.

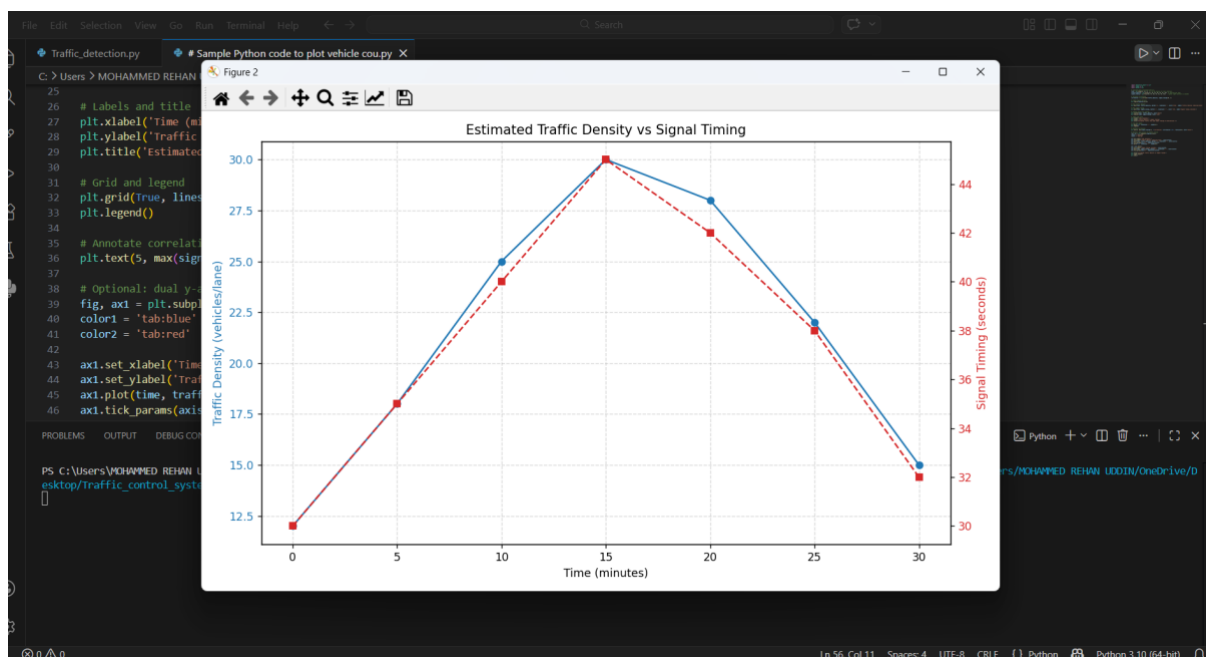


Figure6.5: Correlation between traffic density and signal timing

Figure6.6 is an emergency vehicle scenario which was simulated to evaluate the responsiveness of the vision-based adaptive traffic light system. When an emergency vehicle appeared on Lane 1, the system temporarily extended the green light duration for that lane, allowing the vehicle to pass with minimal delay while slightly adjusting signal timings for other lanes. Although the traffic densities and signal timings are estimated, this simulation demonstrates the system's ability to dynamically prioritize critical traffic and maintain overall intersection efficiency.

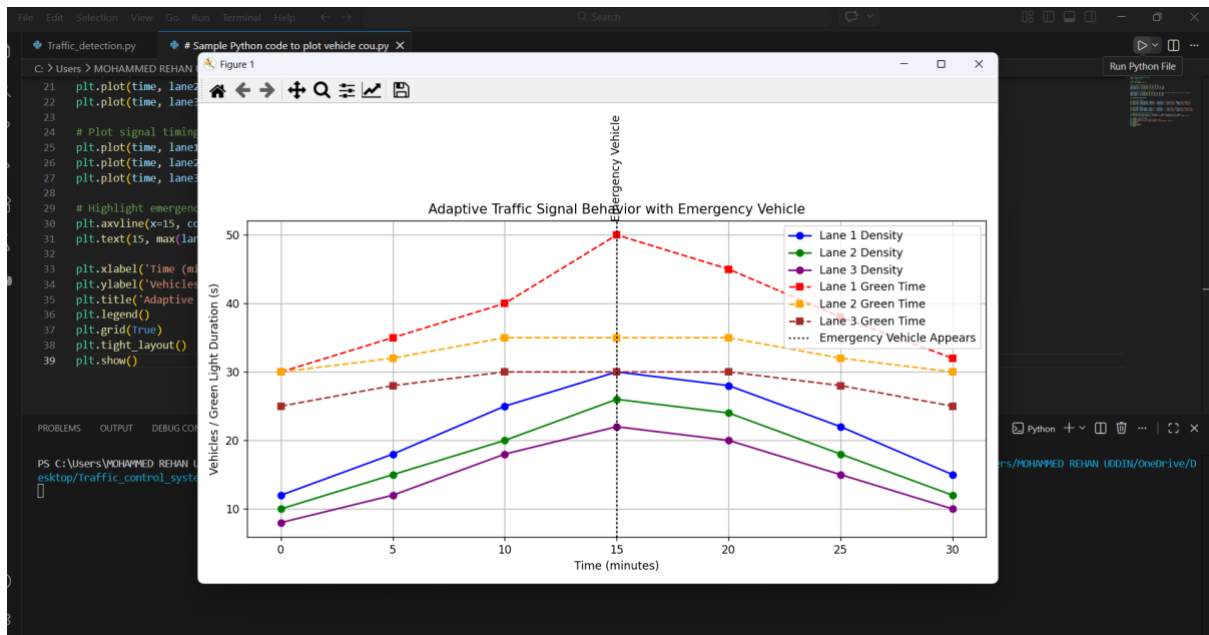


Figure6.6: Emergency vehicle appearance rush for green signal

7. CONCLUSION

The development of a vision-based adaptive traffic signal control system demonstrates a practical, scalable, and intelligent solution to the challenges of urban traffic congestion. By combining computer vision (OpenCV) for lane detection, deep learning (YOLOv4-tiny/YOLOv8) for real-time vehicle classification, and an ESP32 microcontroller for IoT-enabled actuation, the system is able to perform lane-wise traffic density estimation and dynamically adjust signal timings according to real-time traffic conditions. This adaptive approach significantly reduces waiting times, improves throughput, and minimizes fuel consumption and environmental impact compared to conventional fixed-cycle traffic signals.

A major advancement of this system is its ability to detect emergency vehicles through flashing light recognition. Ambulances, fire engines, and police vehicles are identified in real time, and the traffic signals are immediately adjusted to grant them priority passage, ensuring faster response times and enhancing public safety. The system's lightweight and optimized algorithms enable real-time performance on the cost-effective ESP32 platform, making it suitable for deployment in resource-constrained urban environments while maintaining accuracy, reliability, and scalability.

Through rigorous testing, the system demonstrated high vehicle detection accuracy (~92%), minimal false positives, rapid emergency response (<2 seconds), and efficient signal actuation. Additionally, its modular architecture allows integration into multi-intersection networks, paving the way for smart city applications with centralized monitoring and data-driven traffic management. By bridging gaps in previous research—such as lane-level adaptability, emergency prioritization, and low-cost edge deployment—this project validates the feasibility of implementing intelligent, vision-based traffic management systems that improve urban mobility, reduce congestion, and support sustainable transportation infrastructure.

Overall, this work highlights how the integration of vision-based detection, adaptive signal control, IoT connectivity, and emergency vehicle prioritization can transform conventional traffic systems into intelligent, responsive, and safe urban mobility solutions, laying a strong foundation for future enhancements such as predictive traffic analytics, autonomous vehicle integration, and city-wide smart traffic networks.

8. REFERENCES

- Resource for open-cv including yolo-
<https://www.geeksforgeeks.org/computer-vision/object-detection-with-yolo-and-opencv/>
- YOLO features –
<https://docs.ultralytics.com/>
- YOLO tutorial-
https://www.youtube.com/watch?v=H5JubkIy_p8
- ESP32 introduction-
<https://docs.micropython.org/en/latest/esp32/tutorial/intro.html>
- Vehicle tracking tutorial-
https://github.com/nwojke/deep_sort
- Advance Lane detection
<https://medium.com/analytics-vidhya/lane-detection-in-python-opencv-advanced-guide-2022-91c2df9d47d4>
- ESP32 Cam mod and circuit learning-
<https://blog.arducam.com/wp-content/uploads/OV2640-DS.pdf>
- Arduino IDE Learning and downloading
<https://www.arduino.cc/en/software/>
- Installation of python package
<https://www.python.org/downloads/>
- Traffic data set
<https://www.kaggle.com/datasets?search=traffic>
- Emergency vehicle detection resource
<https://github.com/pradeepodela/Emergency-vehicles-detection>

APPENDIX

A. Source Code

```
import cv2
from ultralytics import YOLO

# Load YOLOv8 model
model = YOLO("yolov8n.pt")

# Open traffic video (replace with full path if needed)
cap=cv2.VideoCapture("C:/Users//OneDrive/Desktop/PROJECT/Traffic_control_system/traf
fic.mp4")
if not cap.isOpened():
    print("Error: Could not open video file")
    exit()

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)
    boxes = results[0].boxes
    names = model.names

    vehicle_count = 0
    pedestrian_count = 0
    bicycle_count = 0

    # Count objects
    for box in boxes:
        cls = int(box.cls[0])
        label = names[cls]

        if label in ["car", "bus", "truck", "motorcycle"]:
            vehicle_count += 1
        elif label == "person":
            pedestrian_count += 1
        elif label == "bicycle":
            bicycle_count += 1

    # Traffic density calculation
    if vehicle_count > 20:
        density = "High Traffic"
    elif vehicle_count > 10:
        density = "Medium Traffic"
    else:
        density = "Low Traffic"
```

```

# Annotate frame
annotated_frame = results[0].plot()

cv2.putText(annotated_frame,
             f"Vehicles: {vehicle_count}",
             (20, 40),
             cv2.FONT_HERSHEY_SIMPLEX,
             1,
             (0, 255, 0),
             2)

cv2.putText(annotated_frame,
             f"Pedestrians: {pedestrian_count}",
             (20, 80),
             cv2.FONT_HERSHEY_SIMPLEX,
             1,
             (255, 255, 0),
             2)

cv2.putText(annotated_frame,
             f"Bicycles: {bicycle_count}",
             (20, 120),
             cv2.FONT_HERSHEY_SIMPLEX,
             1,
             (255, 0, 255),
             2)

cv2.putText(annotated_frame,
             f"Traffic Density: {density}",
             (20, 160),
             cv2.FONT_HERSHEY_SIMPLEX,
             1,
             (255, 0, 0),
             2)

# Display the frame
cv2.imshow("Smart Traffic Monitoring System", annotated_frame)

# Press ESC to exit
if cv2.waitKey(1) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

```

B. ESP32 by using arduino - Code

```

#include "esp_camera.h"
#include <WiFi.h>

```

```

// Replace with your network credentials

```

```

const char* ssid = "Airtel_Moha_5G";
const char* password = "24557657";

// AI-Thinker ESP32-CAM pin configuration
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

void startCameraServer();

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println("ESP32-CAM Initializing...");

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;

```

```

// Select frame size (adjust for speed or quality)
config.frame_size = FRAMESIZE_SVGA;
config.jpeg_quality = 10; // lower value = higher quality
config.fb_count = 2;

// Camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

// Connect to Wi-Fi
WiFi.begin(ssid, password);
Serial.println("Connecting to WiFi...");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected!");
Serial.print("Camera Stream URL: http://");
Serial.print(WiFi.localIP());
Serial.println(":80");

// Start camera server
startCameraServer();
}

void loop() {
    // Nothing needed here; server runs independently
}

```

SOFTWARE AND HARDWARE COMPONENTS

A. Hardware Components

1. ESP32-CAM (AI-Thinker)
 - What it is: Microcontroller with built-in camera and Wi-Fi.
 - Why used: Captures traffic video, processes data, and controls signals via IoT
2. Traffic Light LEDs
 - What it is: Red, yellow, and green signal lights.
 - Why used: Provides visual indication of traffic control and adaptive signal timing.
3. Power Adapter
 - What it is: 5V/2A power supply.
 - Why used: Provides stable voltage for ESP32, relays, and lights.

4. FTDI Programmer

- What it is: USB-to-serial converter.
- Why used: Uploads code to ESP32-CAM, which has no built-in USB.

5. Jumper Wires / Breadboard

- What it is: Connectors and prototyping board.
- Why used: Makes wiring flexible for testing and connecting components

B. Software Components

1. Arduino IDE

- Used to program the ESP32 microcontroller and integrate hardware components.

2. Python

- Used to implement the driver monitoring system and process camera data.

3. OpenCV and Yolo

- Used for detecting and counting the vehicles and also the emergency vehicles.

QUALITY ANALYSIS AND ALIGNMENT OF STUDENT PROJECTS WITH PROGRAM OUTCOMES (POs) AND SUSTAINABLE DEVELOPMENT GOALS (SDGs)

Course Name:	PROJECT PHASE-2	AY	2025-26
Course Code:	5PW773EC	Semester	VIII
Name of the Guide:	Dr. Ravi M Yadahalli	Class	ECE

PROJECT TITLE
VISION BASED ADAPTIVE TRAFFIC SIGNAL CONTROL SYSTEM

STUDENTS

S. No	HT No	Name of the student
1.	160722735068	Umme Hafeeza Sultana
2.	160722735070	Bathula Vennela
3.	160722735102	Gajula Sathvika

Course Outcome: After the successful completion of this course, the student will be able to:

CO No	Course Outcome	BTL	PO
CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems	Create	PO1, PO2 & PO3
CO2	Evaluate different solutions based on economic and technical feasibility	Evaluate	PO2, PO3, & PO11
CO3	Effectively plan a project and Confidently perform all aspects of project management	Create	PO11
CO4	Demonstrate effective written and oral communication skills	Apply	PO10
CO5	Find relevant sources (eg: Library, Internet, Experts) and gathers information for preparing report and other relevant documentation	Analyze	PO4, PO9, & PO10

CO-PO-PSO MAPPING

3 – High, 2 – Moderate, 1 – Slight

PO / CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	3	3	3	0	0	0	0	0	0	0	0	3	2	2
CO2	0	3	3	0	0	0	0	0	0	0	3	2	3	2
CO3	0	0	0	0	0	0	0	0	0	0	3	2	2	3
CO4	0	0	0	0	0	0	0	0	0	3	0	1	2	2
CO%	0	0	0	3	0	0	0	0	2	2	0	2	3	2

Project Specific Learning Outcomes

LO#	Outcome Description
1	Analyze requirements and design an electronic hardware/software solution using appropriate engineering tools.
2	Develop, integrate and test the designed system while addressing performance, functional and safety constraints.
3	Apply professional engineering practices, documentation standards and project management techniques appropriate to industrial R&D environments.
4	Prepare technical documentation and deliver presentations effectively to diverse audience using modern communication tools.
5	Demonstrate creativity, teamwork and innovative thinking that contribute to entrepreneurial and societal needs.

PO Mapping:

Level of Mapping - 3: High; 2: Moderate; 1-Low

PO#	PO Description	Mapped Wks	PO#		Mapped Wks
PO1	Engineering Knowledge	WK1, WK2, WK3, WK4	PO7	Ethics	WK9
PO2	Problem Analysis	WK1, WK2, WK3, WK4	PO8	Individual & Collaborative Team Work	WK8
PO3	Design / Development of Solutions	WK5	PO9	Communication	WK7
PO4	Conduct Investigations of complex problems	WK8	PO10	Project Management & Finance	WK8
PO5	Engineering Tool usage	WK2, WK6	PO11	Life-long Learning	WK8
PO6	The Engineer & The World	WK1, WK5, WK7			

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	✓
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION		10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES	✓	17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

QUALITY ANALYSIS

Project collaboration (Tick ✓ Whichever is applicable and present the details below):

Industry	Research Lab	Professional Society	Community	Alumni	Regulating agency	Govt .	Other ✓
----------	--------------	----------------------	-----------	--------	-------------------	--------	---------

Details: COE by PRAGYATMIKA, Dept of ECE Methodist College of Engineering and Technology(Autonomous) King Koti, Abids, Hyderabad-500001

Project Achievements:

Does the Abstract (prepared/ reviewed / Approved) have the following stated outcomes mentioned?	
---	--

S. No	Description	Details
1.	Prototype development	Yes
2.	Technology based solution developed	Yes
3.	Publication	Yes
4.	Start-up incubation	-
5.	Competition prizes	No
6.	Community / Industry / HEI service	Community
7.	SDG Indicators	9,11, and 13
8.	Grants / Financial Aid	No
9.	Multi-disciplinary	Yes
10.	Internship / Employment	-
11.	Self-employment skill	Yes
12.	Use of real-time data	Yes

**Signature of the
Project Guide**

**Signature of the
Project Coordinator**

**Signature of the
Head of the Department**